Palatial Pools Technical Assistance Portal
Proprietary Technology Documentation
Classification: Proprietary & Confidential
Engineered by: Kieran Goldsmith | Palatial Pool Services Pty Ltd

---

Table of Contents

1. [Introduction](#introduction)
2. [System Architecture](#system-architecture)
3. [Ticket Lifecycle and Workflow](#ticket-lifecycle-and-workflow)
4. [Data Models and Formats](#data-models-and-formats)
5. [Security Architecture](#security-architecture)
6. [Infrastructure and Integration](#infrastructure-and-integration)
7. [Performance and Scalability](#performance-and-scalability)
8. [Compliance and Standards](#compliance-and-standards)
9. [Appendices](#appendices)

Introduction

The **Palatial Pools Technical Assistance Portal** (hereafter referred to as the TAP) is a purpose-built IT support and ticketing system designed to streamline issue reporting, feature requests, and resolution tracking within the Palatial Pools ecosystem. The system is engineered to replace informal, ad-hoc communication methods with a structured, auditable, and efficient workflow that supports all stages of technical issue management.

The TAP supports the pool service industry by enabling rapid response times, improved tracking of technical problems, and enhanced customer satisfaction through transparent communication. This document provides a comprehensive technical overview of TAP’s design, architecture, protocols, and operational mechanisms.

System Architecture

Overview

The TAP is architected as a multi-tier web application with modular components to handle user interface, application logic, data persistence, and integration services. It is designed for deployment within Palatial Pools’ secure cloud infrastructure, leveraging containerization and

```
microservices principles for scalability and maintainability.

### High-Level Components

- **User Interface (UI):** Web-based portal accessible via modern browsers supporting TLS 1.3 encrypted HTTPS connections.
- **API Layer:** RESTful services implementing JSON data interchange, facilitating both UI and external integrations.
- **Business Logic Engine:** Handles ticket routing, priority classification, notifications, and SLA tracking.
- **Data Storage:** Relational database for ticket metadata; Object storage for file attachments.
- **Security Services:** Role-based access control (RBAC), encryption, and virus scanning modules.
- **Integration Adapters:** Connectors for external systems, notably the *Gods Eye* platform for operational monitoring.

### Architectural Diagram

```mermaid
graph TD
    A[User Browser] -->|HTTPS/TLS 1.3| B[Load Balancer]
    B --> C[API Gateway]
    C --> D[Business Logic Engine]
    D --> E[(Relational DB - Tickets & Metadata)]
    D --> F[(Object Storage - File Attachments)]
    D --> G[Security Services]
    D --> H[Integration Adapter: Gods Eye]
    H --> I[Gods Eye Monitoring System]

    subgraph Security
        G
    end
```

Ticket Lifecycle and Workflow

Ticket Submission

Users initiate the process by submitting a ticket via the TAP UI or API. Each ticket must include:
```

- **Category:** Predefined taxonomy (e.g., Hardware, Software, Network, Feature Request).
- **Priority Level:** Enum — Low, Medium, High, Critical.
- **Description:** Detailed text outlining the issue or request.
- **Attachments:** Optional file uploads (images, logs, error dumps).

Ticket Routing and Assignment

Upon submission, the Business Logic Engine classifies the ticket using rule-based algorithms:

- **Category Routing:** Tickets are automatically assigned to the appropriate team or technician group.
- **Priority-based Escalation:** Critical tickets trigger immediate notification workflows and SLA timers.
- **Load Balancing:** Assignments consider current workload using a weighted round-robin algorithm.

Status Tracking and Updates

Tickets progress through defined states:

```
[Open] -> [In Progress] -> [Waiting on Customer] -> [Resolved] -> [Closed]
```

Status changes trigger notifications to relevant stakeholders via email and TAP portal messages. All updates are timestamped and logged.

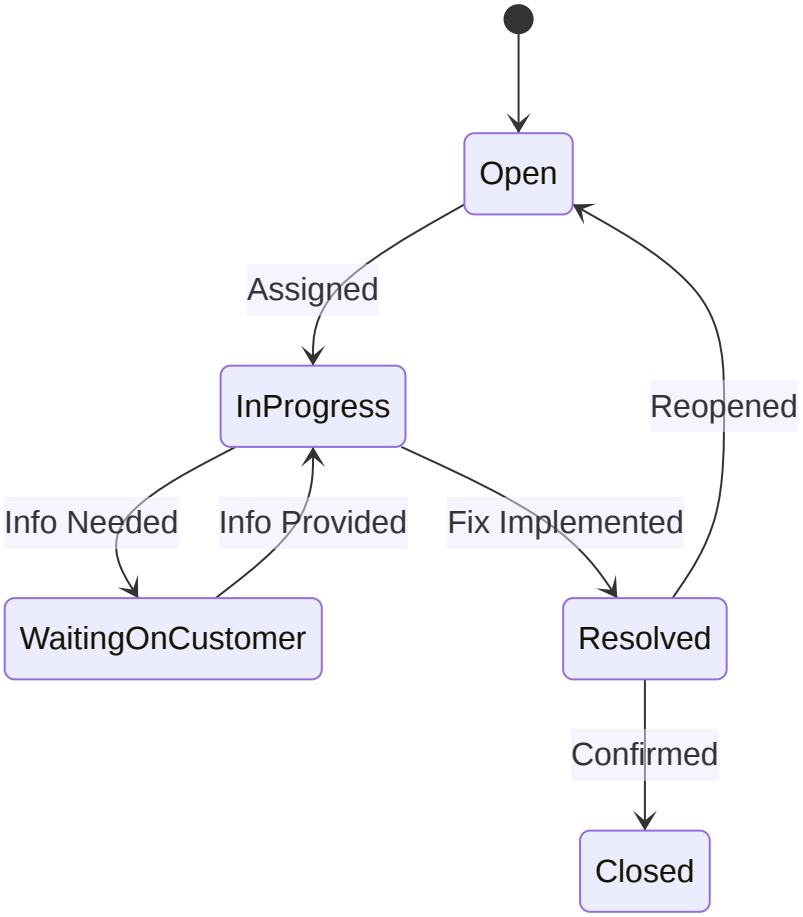
Resolution Metrics

The system collects metrics including:

- Time-to-first-response
- Time-to-resolution
- Reopen rate
- Customer satisfaction scores (via post-resolution surveys)

These metrics feed into continuous improvement programs.

Workflow Flowchart



Data Models and Formats

Ticket Data Model

Field	Type	Description
ticket_id	UUID	Unique identifier for the ticket
created_at	ISO 8601 DateTime	Timestamp of ticket creation
updated_at	ISO 8601 DateTime	Timestamp of last update
category	Enum	Ticket category
priority	Enum	Priority level
status	Enum	Current status
description	Text	Detailed description
submitter_id	UUID	User ID of ticket submitter
assigned_group_id	UUID	Team or technician group ID assigned
attachments	Array of URIs	References to stored file attachments
resolution_notes	Text	Notes related to ticket resolution
sla_due_time	ISO 8601 DateTime	SLA target resolution time

API Request/Response Example

Create Ticket Request:

```
{
  "category": "Hardware",
  "priority": "High",
  "description": "Pump motor failure detected, error code E45.",
  "attachments": [
    "..."
  ]
}
```

Create Ticket Response:

```
{
  "ticket_id": "a3f1e9bc-8f4e-4d2e-9bc8-1a57f3d2d3a5",
  "status": "Open",
  "created_at": "2024-05-15T08:42:00Z"
}
```

Security Architecture

Data Encryption

- **Data at Rest:**
  All ticket data stored in the backend uses AES-256 encryption. Object storage for file attachments is encrypted using server-side encryption compliant with AWS S3 AES-256 standards or equivalent.

- **Data in Transit:**
  All client-server communications employ TLS 1.3 to prevent eavesdropping and MITM attacks.

Access Control

- **Role-Based Access Control (RBAC):**
  Access to tickets and system functions is restricted based on user roles:
  - *User:* Can submit and view own tickets.
  - *Technician:* Can view and update assigned tickets.
  - *Admin:* Full system access including user management.

- **Authentication:**
  Integration with corporate identity providers using OAuth 2.0 and OpenID Connect (OIDC) protocols.

File Upload Security

- All file uploads undergo virus scanning using industry-standard engines (e.g., ClamAV or commercial equivalents).
- Files exceeding defined size limits (e.g., 25 MB) are rejected to prevent DoS attacks.

Audit and Logging

- Immutable audit logs capture all ticket state transitions and user actions, stored securely for compliance and forensic purposes.

Infrastructure and Integration

Deployment Environment

- TAP is deployed on a container orchestration platform (e.g., Kubernetes) within Palatial Pools' private cloud or approved public cloud providers with data residency in Australia.
- Uses CI/CD pipelines with automated testing and vulnerability scanning.

Integration with Gods Eye

- Gods Eye is Palatial Pools' operational monitoring platform. TAP integrates via secure REST APIs.
- Key integration points include:
  - **Incident Correlation:** TAP tickets linked to sensor alerts from Gods Eye.
  - **Real-Time Status Sync:** Status updates pushed to Gods Eye dashboard.
  - **User Authentication:** Shared SSO credentials.

File Storage and Attachment Handling

- Attachments are stored in scalable object storage solutions supporting redundancy and backups.

- URL references include time-limited signed URLs for secure access.

Priority-Based Routing Algorithm (Simplified Pseudocode)

```
function assignTicket(ticket):
    candidates = getTechniciansByCategory(ticket.category)
    sortedCandidates = sortByWorkload(candidates)

    if ticket.priority == "Critical":
        notifyAll(candidates)
        assigned = sortedCandidates[0]
    else:
        assigned = sortedCandidates[0]

    assign(ticket, assigned)
```

Performance and Scalability

- Designed for concurrent handling of thousands of tickets with sub-second response times on common API calls.
- Horizontal scaling of stateless API servers allows dynamic resource allocation during peak periods.
- Database employs indexing on critical fields (ticket_id, status, priority) to optimize query performance.
- SLA timers and notification engines are implemented using event-driven architecture (e.g., Apache Kafka or RabbitMQ message queues).

Compliance and Standards

- **Australian Consumer Law (ACL):**
  TAP is designed to provide transparent, factual information regarding service issues and resolution times. It does not make misleading claims and supports consumer guarantees by providing auditable records of service requests and responses.

- **Data Privacy:**
  Compliance with the Australian Privacy Principles (APPs) under the Privacy Act 1988, ensuring personal information is handled securely and with consent.
- **Security Standards:**
  Implementation adheres to OWASP Top 10 for web application security and NIST SP 800-53 guidelines for access control and encryption.
- **Networking Protocols:**
  Use of IEEE 802.11 standards for wireless connectivity within service locations, ensuring secure Wi-Fi communication when TAP is accessed on-site.

Appendices

Appendix A: Technical Glossary

Term	Definition
AES-256	Advanced Encryption Standard with 256-bit keys
OAuth 2.0	Authorization framework enabling delegated access
OpenID Connect (OIDC)	Identity layer on top of OAuth 2.0 for user authentication
TLS 1.3	Transport Layer Security protocol version 1.3 for encryption
RBAC	Role-Based Access Control system

Appendix B: Sample Ticket JSON Schema (Simplified)

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "Ticket",
  "type": "object",
  "properties": {
    "ticket_id": { "type": "string", "format": "uuid" },
    "category": { "type": "string", "enum": ["Hardware", "Software",
"Network", "Feature Request"] },
    "priority": { "type": "string", "enum": ["Low", "Medium", "High",
"Critical"] },
    "status": { "type": "string", "enum": ["Open", "In Progress", "Waiting
on Customer", "Resolved", "Closed"] },
    "description": { "type": "string" },
    "created_at": { "type": "string", "format": "date-time" },
    "updated_at": { "type": "string", "format": "date-time" },
    "attachments": {
      "type": "array",
      "items": { "type": "string", "format": "uri" }
    }
  },
  "required": ["ticket_id", "category", "priority", "status", "description",
"created_at"]
}
```

Document Control:

This document is proprietary and confidential. Distribution and reproduction are restricted to authorized personnel of Palatial Pool Services Pty Ltd.

”`